

The Honjo Masamune Language: A Cut-Primitive Domain-Specific Language for Cheminformatics by Individuation

Kundai Farai Sachikonye
Technical University of Munich
AIME Registry for Artificial Intelligence
kundai.sachikonye@wzw.tum.de

August 19, 2026

Abstract

We specify *Honjo Masamune* (`honjo`; source extension `.hj`), a domain-specific language for cheminformatics in which the sole computational primitive is the *cut*: the act of individuating a part from the rest of a bounded whole at a strictly positive, irreducible cost—the *floor* $\beta > 0$. The language is built on a single semantic identity carried from its underlying theory: generating an atom, forming a chemical bond, and tracking an item through a process are not distinct operations but the same cut operation at different arities. A length-one cut individuates an atom; a cut between two items is a bond; a residue-chained sequence of cuts is a process track. `honjo` therefore has one verb and one composition rule, from which atom generation, compound construction, molecular geometry, and causal process tracking all follow.

We give: (i) the semantic model—a finite weighted *contact graph* on which every `honjo` value is a cut of accountable residue, never a bare number; (ii) a complete lexical and context-free grammar in EBNF; (iii) a static *accountability* type system in which every value carries both the floor at which it was cut and the *provenance* of the data it was cut from, so that a value claiming zero residue is rejected at compile time and a value computed over supplied data can never be mistaken for one computed over stated data; (iv) a small-step operational semantics in which evaluation *is* measurement, the committed-cut count is monotone, and provenance is monotone, giving every program an intrinsic clock and an intrinsic audit trail; (v) a *verdict* discipline under which every operation that can fail returns a labelled outcome rather than an empty value, with a proof that a value-or-nothing interface conflates four distinguishable failures no caller discipline recovers; (vi) a standard library of verbs, each bound by name to an operation of the underlying framework, extended here with committed-cell counts on bonds and a delocalised-system form, without which multiply-bonded and aromatic structures are not representable; and (vii) a two-target compilation model with a *conditional* equivalence theorem: the two back ends agree up to the floor exactly when the target’s numeric resolution is at least as fine as the floor, a hypothesis we make checkable rather than assume.

Structures enter `honjo` through a separate converter, which supplies contact graphs carrying per-element provenance; `honjo`’s contribution is what happens after, and the interface between them is specified here as a typed import that refuses rather than silently accepting a graph whose provenance does not meet a program’s stated requirement. We state what the language does not do, including three limitations of the underlying chemistry it inherits and cannot repair.

Keywords: domain-specific language; cheminformatics; individuation; cut primitive; contact graph; accountability type system; provenance; verdict; operational semantics; minimum cut; two-target compilation.

Contents

1	Introduction	3
1.1	Papers as programs	3
1.2	One primitive: the cut	3
1.3	Three properties the language enforces	3
1.4	Where structures come from	4
1.5	Two targets, one language	4
1.6	What this document specifies	4
2	The semantic model	5
2.1	The contact graph	5
2.2	Provenance	5
2.3	Measurement, not simulation	6
3	Lexical structure	6
4	Grammar	7
5	The accountability type system	9
5.1	Types	9
5.2	The positivity rule	9
5.3	Cut typing	9
5.4	Provenance is monotone	10
5.5	Soundness	10
6	Verdicts	11
6.1	The verdict set	11
6.2	The alternative interface cannot express this	12
7	Operational semantics	12
7.1	Cut reduction	12
7.2	Track reduction and convergence	13
8	The standard library	13
8.1	<code>cut</code> — atom individuation	13
8.2	<code>~</code> — bonding with committed cells	13
8.3	<code>deloc</code> — delocalised systems	14
8.4	<code>close</code> — compound by closure	14
8.5	<code>track</code> — accountable propagation	14
9	Importing structures	14
10	Core IR and the two-target model	15
10.1	The core IR	15
10.2	Target R (Rust): cuts as minimum cuts	16
10.3	Target T (TypeScript/WebGL): cuts as render passes	16
10.4	Target equivalence, conditionally	16
11	Worked programs	17
11.1	Generate an atom	17
11.2	Build a compound, with committed cells	17
11.3	A delocalised system	17
11.4	Import a structure and require its provenance	18

12 Implementation requirements	18
12.1 Conformance suite	18
13 What honjo does not do	19
14 Conclusion	20

1 Introduction

1.1 Papers as programs

The framework underlying this language consists of a sequence of results that are, in form, *operations*: a procedure that individuates an atom from spectroscopic structure; a procedure that decides whether two atoms bond and in what geometry; a procedure that tracks an item’s uncertainty through a process and returns an accountable account of what happened to it. Each such result is already, in effect, a program with a proof of correctness attached. This specification makes that literal: it defines a language whose statements *are* those operations, so that a paper becomes a script and a derivation becomes a run.

The design discipline throughout is that we implement only what is already established. Every verb of the language binds to an operation of the framework that has an explicit construction; the language adds composition, typing, and execution, not new chemistry. Where the underlying chemistry is qualitative—as it is for bond energetics—the language is qualitative too, and Section 13 says so rather than letting the type system imply a precision the operations do not have.

1.2 One primitive: the cut

The language has a single computational primitive, the *cut*, and a single reason for it. To individuate any part of a bounded whole—to make it a distinct thing—costs a strictly positive, irreducible minimum, the floor $\beta > 0$. This one fact is the engine of the underlying theory, and it is the engine of the language. Every honjo operation is a cut:

- A *generation* is a cut of arity one: individuating an atom of atomic number Z from the medium (Section 8.1).
- A *bond* is a cut of arity two: the single shared boundary between two items, which exists iff joining lowers total boundary thickness, and which commits a definite number of valence cells (Section 8.2).
- A *compound* is a closure of cuts: cuts continued until every constituent’s vacancy is driven to zero (Section 8.4).
- A *track* is a residue-chained sequence of cuts: the residue of each cut is the cause of the next, and the chain is the propagation of an item’s uncertainty through a process (Section 8.5).

Generation is not a different verb from tracking; it is a track of length one. This is the central design decision and it is forced by the theory, not chosen for convenience: in the underlying framework a reaction step and a measurement step are the same cut, and generation, bonding, and tracking are that cut at arities one, two, and many. honjo therefore has one verb (*cut*) and one composition rule (residue chaining); all other surface forms are sugar over these.

1.3 Three properties the language enforces

Beyond the cut, the specification enforces three properties, each of which exists because its absence produces a specific, identifiable defect.

- (P1) **No sharp cut.** No value may claim zero residue (Theorem 5.4). The forbidden object—a separation at no cost—is not merely discouraged but untypeable.
- (P2) **No laundered provenance.** A structure that reaches honjo carries, per element, whether a source *stated* it or a converter *supplied* it from a convention. honjo propagates this through every operation (Theorem 5.7), so a result computed over supplied data cannot be reported as though computed over stated data. Without this, the audit performed at the converter boundary is discarded at the first cut.
- (P3) **No empty-as-failure.** Every operation that can fail returns a labelled verdict, and a value appears only under the label that warrants it (Theorem 6.3). A `close` that cannot drive vacancies to zero and a `close` over a closed-shell atom are different outcomes; returning nothing for both conflates them (Theorem 6.4).

1.4 Where structures come from

honjo generates structures from atomic number by `cut`, which suffices for the worked examples of Section 11 and for nothing else: no real corpus is expressed atom-by-atom in source text. Structures enter honjo from a converter, an external component that reads records in whatever formats a store holds and emits contact graphs with per-element provenance.

The interface is specified in Section 9 and is deliberately narrow. honjo does not read files, does not parse line notations, and does not perceive rings or aromaticity; it consumes a contact graph whose elements are already tagged, and it *refuses* a graph whose provenance does not meet the program’s stated requirement. What the converter does is outside this document. That the boundary is typed, and that crossing it cannot silently upgrade supplied data to stated data, is inside it.

1.5 Two targets, one language

honjo exists in two parallel implementations from a single specification:

1. A **TypeScript** compiler and WebGL 2 runtime for the interactive online tool. Here a cut is realised as a GPU render pass: the framebuffer that results from rendering *is* the measurement. This is the fast, exploratory, in-browser version.
2. A **Rust** compiler and native runtime for the reference and production version. Here a cut is realised as an exact maximum-flow minimum-cut computation on the contact graph. This is the authoritative version against which the online tool is checked.

Both share one front end (lexer, parser, type checker) and one typed core intermediate representation (Section 10); they differ only in the back end that realises a cut. Theorem 10.5 states the conditions under which the two agree, and Theorem 10.6 is explicit that the equivalence is conditional on a property of the target hardware which the specification requires to be measured rather than assumed.

1.6 What this document specifies

Section 2 fixes the semantic model: the contact graph, the cut, the floor, the residue, provenance, and accountability. Section 3 gives the lexical grammar; Section 4 the context-free grammar in EBNF. Section 5 defines the accountability type system and proves its soundness and provenance monotonicity. Section 6 defines the verdict discipline and proves the conflation theorem. Section 7 gives the small-step operational semantics and proves cut monotonicity. Section 8 specifies the standard library. Section 9 specifies the converter interface. Section 10 defines the core IR and the two-target model. Section 11 gives worked programs. Section 12 states implementation requirements and the conformance suite. Section 13 states what the language does not do.

2 The semantic model

honjo is defined over a single mathematical object, the contact graph, and a single operation on it, the cut. We fix these before any syntax.

2.1 The contact graph

Definition 2.1 (Contact graph). A *contact graph* is a finite weighted graph $\Gamma = (V, E, w)$ with a distinguished *medium* vertex $\mathbf{m} \in V$ adjacent to every other vertex, and $w: E \rightarrow \mathbb{R}_{>0}$ a strictly positive weight. Vertices $v \neq \mathbf{m}$ are *items*; an edge between two items is a *contact*; $w(e)$ is the cost of the separation it maintains. The graph is the runtime state of a honjo program: every value the program holds is a structure on Γ .

Definition 2.2 (Floor). The *floor* of a contact graph is a constant $\beta > 0$ with $w(e) \geq \beta$ for every edge. No separation is free. The floor is a program-level parameter (set by `floor`, Section 4); its positivity is invariant and enforced by the type system (Section 5).

Definition 2.3 (Cut). A *cut* of an item set S ($\emptyset \neq S \subsetneq V$, $\mathbf{m} \notin S$) is the edge boundary $\partial(S) = \{e \in E : |e \cap S| = 1\}$, of weight $w(\partial(S)) = \sum_{e \in \partial(S)} w(e) \geq \beta$. The cut individuates S from the rest. A cut *event* commits one such boundary: it fixes the edges of $\partial(S)$ as realised separations and advances the program’s cut count by one.

Lemma 2.4 (Cut weights are positive). *For every admissible S , $w(\partial(S)) \geq \beta > 0$.*

Proof. S contains an item v and omits $\mathbf{m}$, and $\{v, \mathbf{m}\} \in E$ by Theorem 2.1, so $\{v, \mathbf{m}\} \in \partial(S)$ and the cut is non-empty. Every edge weight is at least β by Theorem 2.2, so the sum over a non-empty edge set is at least β . $\square$

Theorem 2.4 is why medium-adjacency is required rather than merely natural: it guarantees every cut is non-empty, hence that Theorem 5.4 has content for every value the language can form.

Definition 2.5 (Residue). The *residue* of a cut event is the weight $\rho = w(\partial(S)) \geq \beta$ it deposits as committed boundary. The residue is the value the cut produces and the cause of any subsequent cut: a chain of cuts is one in which each residue is consumed as the boundary condition of the next (Section 8.5).

2.2 Provenance

A contact graph reaching honjo from a converter carries, per element, a record of where that element came from. The language propagates it.

Definition 2.6 (Provenance tags). The provenance tag set is $P = \{\text{stated}, \text{supplied}\}$, ordered $\text{stated} < \text{supplied}$, with the readings: **stated**—the originating record asserts this element; **supplied**—it was produced by applying a declared convention to the record’s silence. A *provenance map* π assigns a tag to each vertex and edge of a contact graph. Elements neither stated nor suppleable are not present in the graph and therefore carry no tag.

Definition 2.7 (Provenance of a cut). The provenance of a cut event on S is

$$\pi(\partial(S)) = \max_{e \in \partial(S)} \pi(e),$$

the maximum under $\text{stated} < \text{supplied}$: a cut is **stated** exactly when every edge it severs is **stated**, and **supplied** as soon as any one of them is.

Principle 2.8 (Accountability). Every honjo value is a cut, and every cut carries its residue, its floor, and its provenance. There are no bare numbers in honjo: a quantity is always paired with the floor at which it was individuated and the provenance of the data it was cut from. A value claiming residue below the floor, or zero residue, is not a value—it is the forbidden sharp cut, and the type system rejects it (Theorem 5.4). A value cut from supplied data is not a value of stated provenance, and the type system will not report it as one (Theorem 5.7).

Remark 2.9 (Why provenance is in the type and not in a side channel). A converter that audits its own conventions produces, at some cost, a statement about how much of a structure is record and how much is convention. If the consuming language discards that statement at import, the audit was performed for nothing: every downstream result is again indistinguishable from one computed over fully stated data. Carrying the tag in the value’s type is what makes discarding it impossible rather than merely bad practice, and it is the same reason the floor is in the type rather than in a comment.

Remark 2.10 (Why a graph and not a number line). honjo computes on a contact graph rather than on real numbers because the quantities it manipulates—atomic identity, bond existence, process accountability—are minimum cuts, not scalars. A bond’s strength is the weight of a shared boundary; an atom’s identity is the invariant minimum cut against the medium; a track’s result is an accountable chain of cuts. Representing these as graph cuts (rather than as fitted scalars) is what lets the same primitive serve generation, bonding, and tracking, and is what the two back ends each realise: exactly (Rust, max-flow) or by rendering (TypeScript, GPU).

2.3 Measurement, not simulation

Principle 2.11 (Evaluation is measurement). Executing a honjo cut performs the individuation; it does not simulate a description of it. Operationally (Section 7) this means a cut evaluation mutates the contact graph—it commits boundary and advances the count—rather than returning a pure function of inputs. The distinction is observable: re-evaluating a cut is a new cut at a higher count, never a cached recomputation (Theorem 7.2).

Remark 2.12 (What Theorem 2.11 does not claim). The principle is about the language’s evaluation model, not about physical instrumentation: running `cut 8` performs the individuation *the framework operation defines*, and the accuracy of that operation as a description of oxygen is a property of the operation, not of honjo. The claim is that honjo does not memoise it, not that honjo has measured anything in a laboratory.

3 Lexical structure

A honjo source file is UTF-8 text with extension `.hj`. Lexing produces a token stream from the following classes.

Definition 3.1 (Token classes).

- **Comments.** `-` to end of line; discarded.
- **Keywords.** `floor`, `cut`, `close`, `track`, `until`, `yield`, `when`, `emit`, `observe`, `in`, `as`, `let`, `medium`, `converge`, `diverge`, `with`, `by`, `import`, `module`, `export`, `assert`, `ring`, `deloc`, `refuse`, `require`, `unless`, `stated`, `on`.
- **Identifiers.** `[A-Za-z_][A-Za-z0-9_]*`, not a keyword.
- **Numbers.** decimal integers and floats with optional exponent: `[0-9]+(.[0-9]+)?([eE][+-]?[0-9]+)?`.
- **Strings.** double-quoted, with the usual escapes.
- **Operators and punctuation.** `:=` (bind), `~` (cut between / bond), `( ) [ ] { } , : . # > < >= <= == != ->`.

Whitespace is insignificant except as a token separator; honjo is not layout-sensitive.

Notation 3.2 (Floors in literals). A numeric literal may carry an explicit floor annotation with the suffix form `value#floor` (read “value at floor”); e.g. `1.0e-7#1e-9`. A literal without an explicit floor inherits the ambient program floor set by the most recent `floor` declaration in scope. There is no way to write a literal of zero floor (Theorem 5.4).

Notation 3.3 (Cell counts on bonds). The bond operator admits a committed-cell annotation with the postfix form `a ~ b : k` for $k \in \{1, 2, 3\}$. Bare `a ~ b` abbreviates `a ~ b : 1`. The form `deloc ring(...)` introduces a delocalised system and is *not* an abbreviation for any assignment of per-bond counts; Theorem 8.5 says why.

4 Grammar

We give the context-free grammar in EBNF ([International Organization for Standardization, 1996](#)). Nonterminals are *italicised*; terminals are `typewriter`; `{x}` is zero or more, `[x]` optional, `|` alternation.

Definition 4.1 (honjo grammar).

```

program ::= { decl }
decl ::= floorDecl | import | moduleDecl | stmt
floorDecl ::= floor number
import ::= import qname | graphImport
graphImport ::= let ident := import graph string
               [require provReq] [unless recovery]
provReq ::= stated | supplied < numlit
recovery ::= refuse | emit string
moduleDecl ::= module ident { { decl } }
stmt ::= bind | track | observe | assert | expr
bind ::= [let] ident := expr
expr ::= cut | bondExpr | delocExpr | closeExpr | call | ident | number | string | ( expr )
cut ::= cut cutArg
cutArg ::= number | ident
bondExpr ::= expr ~ expr [ : cellCount ] [when cond]
cellCount ::= 1 | 2 | 3
delocExpr ::= deloc ring ( argList ) [cells : numlit]
closeExpr ::= close ident ( argList ) [by ident]
track ::= track ident in expr
         [with reps repList]
         until admit yield ident
admit ::= converge | diverge | cond
observe ::= observe expr [as ident]
assert ::= assert cond [emit string]
call ::= qname ( [argList] )
argList ::= arg { , arg }
arg ::= [ident : ] expr
cond ::= expr relop expr | ident . verdict == label
relop ::= > | < | >= | <= | == | !=
repList ::= ident { , ident }
qname ::= ident { . ident }
number ::= numlit [ # numlit ]

```

Remark 4.2 (Reading the core forms). The language is small. `cut N` generates (arity-one cut); `a ~ b : 2 when c` bonds with two committed cells, guarded; `deloc ring(...)` introduces a delocalised system; `close X(...)` drives an item to shell closure; `track x in P until converge yield r` propagates and binds the accountable result. `import graph` brings in a structure from a converter under a stated provenance requirement. `observe` forces measurement of a value; `assert` is a check that may halt. Everything else—`let`, `module`, named arguments—is conventional.

Remark 4.3 (What changed from a bond with no cell count). An earlier form of this grammar had `a ~ b` with no annotation, which makes a double interface and two single interfaces the same

expression and leaves multiply-bonded structures inexpressible. The committed-cell count is not new chemistry: it is the number of valence cells the interface commits, which the underlying bonding operation already computes and which the surface syntax previously had no way to write down. Theorem 8.5 explains why delocalisation needs a different form rather than a fractional count.

5 The accountability type system

honjo’s types record *what kind of cut* a value is, *at what floor*, and *over what provenance*. The system’s purpose is to make Theorem 2.8 statically enforceable.

5.1 Types

Definition 5.1 (Types). The honjo types are

$$\tau ::= \text{Atom} \mid \text{Bond} \mid \text{Deloc} \mid \text{Compound} \mid \text{Path} \mid \text{Scalar} \mid \text{Graph} \mid \text{Cut}.$$

Cut is the supertype of all individuated values; Atom, Bond, Deloc, Compound, Path are its arity- and role-refinements (Atom = arity-one cut, Bond = arity-two cut, Deloc = a cut over a delocalised system, Compound = closure of cuts, Path = residue chain). Scalar is a measured number; it is never bare. Graph is an imported contact graph (Section 9). Every typed value v additionally carries a *floor annotation* $\beta(v) > 0$, a *residue* $\rho(v) \geq \beta(v)$, and a *provenance* $\pi(v) \in P$.

Definition 5.2 (Typing judgement). A judgement $\Gamma \vdash e : \tau @ \beta / p$ reads “in context Γ , expression e has type τ , is individuated at floor $\beta > 0$, and has provenance $p \in P$.” The context carries the ambient floor set by the most recent `floor` declaration and the types, floors and provenances of bound identifiers.

5.2 The positivity rule

Rule 5.3 (Floor positivity). Every value-introducing rule has the side condition $\beta > 0$ and $\rho \geq \beta$. A literal or expression whose annotation forces $\beta \leq 0$ or $\rho < \beta$ is ill-typed.

$$\frac{\Gamma \vdash \text{numlit } n \quad \beta > 0}{\Gamma \vdash n \# \beta : \text{Scalar} @ \beta / \text{stated}}$$

Theorem 5.4 (No zero-residue value). *There is no well-typed honjo expression of residue 0. Equivalently, the sharp cut is not expressible: every typeable value has $\rho \geq \beta > 0$.*

Proof. By induction on typing derivations. The only value introductions are literals (Theorem 5.3, side condition $\beta > 0$, and a literal’s residue is its annotated floor), cuts (residue = $w(\partial(S)) \geq \beta > 0$ by Theorem 2.4), imports (Theorem 9.2, whose side condition requires the imported graph’s weights to satisfy Theorem 2.2), and the derived forms (bond, deloc, close, track) whose residues are sums or chains of cut residues, hence $\geq \beta$. No rule introduces a value of residue $< \beta$, and $\beta > 0$ is a side condition on every floor-introduction. Hence no derivation concludes $\rho = 0$. $\square$

5.3 Cut typing

Rule 5.5 (Generation; bond; delocalisation; close).

$$\frac{\Gamma \vdash Z : \text{Scalar} \quad Z \in \mathbb{N}^+}{\Gamma \vdash \text{cut } Z : \text{Atom} @ \beta_\Gamma / \text{stated}}$$

$$\frac{\Gamma \vdash a : \text{Atom} / p_a \quad \Gamma \vdash b : \text{Atom} / p_b \quad k \in \{1, 2, 3\}}{\Gamma \vdash a \sim b : k : \text{Bond} @ \beta_\Gamma / \max(p_a, p_b)}$$

$$\frac{\Gamma \vdash a_i : \text{Atom} / p_i \quad (i = 1..n) \quad n \geq 3}{\Gamma \vdash \text{deloc ring}(a_1, \dots, a_n) : \text{Deloc} @ \beta_\Gamma / \max_i p_i}$$

$$\frac{\Gamma \vdash X : \text{Atom} / p_X \quad \Gamma \vdash a_i : \text{Atom} / p_i}{\Gamma \vdash \text{close } X(a_1, \dots, a_k) : \text{Compound} @ \beta_\Gamma / \max(p_X, \max_i p_i)}$$

The bond rule additionally requires its guard, when present, to be a Boolean-valued *cond*. Note that cut Z is stated: an atom generated from an atomic number is not supplied from a record's silence, because there was no record.

Rule 5.6 (Track).

$$\frac{\Gamma \vdash x : \text{Atom or Compound} / p_x \quad \Gamma \vdash P : \text{Compound or Path} / p_P}{\Gamma \vdash \text{track } x \text{ in } P \text{ until } \text{admit yield } r : \text{Path} @ \beta_\Gamma / \max(p_x, p_P)}$$

5.4 Provenance is monotone

Theorem 5.7 (Provenance monotonicity). *For every typing rule, the provenance of the conclusion is the maximum under **stated** < **supplied** of the provenances of its premises. Consequently, along any derivation: a value is **stated** only if every value it was derived from is **stated**; and once a value is **supplied**, no operation of the language yields a **stated** value from it.*

Proof. By inspection of Theorems 5.5, 5.6 and 9.2: each conclusion's provenance is written as a maximum over premises, and Theorem 5.3 introduces **stated** with no premises. Since the maximum of a finite set containing **supplied** is **supplied**, and every rule's conclusion is such a maximum, no rule has a **stated** conclusion with a **supplied** premise. Induction on the derivation gives both claims. $\square$

Corollary 5.8 (Supplied data cannot be laundered). *There is no well-typed honjo expression of provenance **stated** whose derivation reads a value of provenance **supplied**. In particular, no sequence of cuts, bonds, closures or tracks converts a supplied structure into a stated result.*

Proof. Immediate from Theorem 5.7: the conclusion's tag is the maximum over premises, and the maximum of any set containing **supplied** is **supplied**. $\square$

Remark 5.9 (Why the maximum and not a count). Theorem 2.7 takes the maximum rather than, say, the fraction of supplied edges. This is deliberate and it is conservative: a cut severing one supplied edge among twenty is **supplied**, because the value depends on that edge and the edge was convention. A finer statistic—what fraction of a result rests on supplied data—is useful and is not a type; Section 9 carries it as an attribute of a **Graph** value, where it can be tested by a program without weakening the guarantee that the tag provides.

5.5 Soundness

Theorem 5.10 (Type soundness). *honjo typing is sound for the operational semantics of Section 7: if $\Gamma \vdash e : \tau @ \beta / p$ and e evaluates to a value v , then v has type τ , floor $\beta > 0$, residue $\rho(v) \geq \beta$, and provenance $\pi(v) = p$ (progress and preservation).*

Proof sketch. Standard syntactic soundness (Milner, 1978; Pierce, 2002). *Progress*: a well-typed closed expression is a value or steps (every form has a reduction rule in Section 7, and every form that can fail has a verdict-producing rule, Section 6, so no well-typed expression is stuck). *Preservation*: each reduction rule preserves the type and the floor annotation; preserves $\rho \geq \beta > 0$ by Theorem 5.4 applied at each step; and preserves provenance because the operational rules compute π by the same maxima as the typing rules (Theorem 5.7). The only non-standard clause is that reductions are state-mutating (Theorem 2.11); preservation holds because the cut count and committed boundary are monotone (Theorem 7.2), so no step lowers a residue below the floor. $\square$

Remark 5.11 (The soundness proof is a sketch and is stated as one). Theorem 5.10 is given as a sketch because the mutation clause makes the standard argument non-routine: preservation must be established against a changing graph rather than a fixed store. We state the shape of the argument and identify the non-standard step; a full proof requires a formalisation we have not carried out, and Section 13 records this.

6 Verdicts

Several honjo operations can fail, and failing in different ways calls for different responses. This section fixes what an operation returns when it cannot return a value.

6.1 The verdict set

Definition 6.1 (Verdict). A *verdict* is a pair (ℓ, π) with label ℓ from

$$L = \{\text{cut}, \text{unclosed}, \text{inert}, \text{nonconvergent}, \text{underprovenanced}, \text{subfloor}\}$$

and a label-specific payload:

Label	Payload
cut	the value: its type, residue, floor and provenance
unclosed	the constituents whose vacancy is not zero, and their ν
inert	the closed-shell participants, certified $\nu = 0$
nonconvergent	the chain reached, and its terminal alignment
underprovenanced	the requirement, and the measured supplied fraction
subfloor	the edge or residue found below β , and β

Definition 6.2 (Assignment). For an operation over inputs I under ambient floor β and a stated provenance requirement R , the verdict is the first applicable clause:

- (V1) if any input weight or residue is below β , return $(\text{subfloor}, \cdot)$;
- (V2) else if $\pi(I)$ does not satisfy R , return $(\text{underprovenanced}, \cdot)$;
- (V3) else if the operation is **close** and every participant has $\nu = 0$, return $(\text{inert}, \cdot)$;
- (V4) else if the operation is **close** and closure leaves some $\nu > 0$, return $(\text{unclosed}, \cdot)$;
- (V5) else if the operation is **track** and the chain does not admit, return $(\text{nonconvergent}, \cdot)$;
- (V6) else return (cut, v) .

The clause order is not arbitrary: each clause precedes every clause whose evidence it would corrupt. (V1) precedes all others because a sub-floor input makes every downstream residue meaningless; (V2) precedes (V3)–(V5) so that a provenance shortfall is never reported as a chemical failure; and (V3) precedes (V4) because an inert participant is not an unclosed one—it has nothing to close.

Proposition 6.3 (Only one verdict carries a value). *Of the six labels, only cut has a payload containing a honjo value. Consequently the presence of a value determines $\ell = \text{cut}$, and the five failure outcomes are distinguishable from it and from each other by label alone.*

Proof. By inspection of the payload table in Theorem 6.1. The **unclosed** payload carries constituents and vacancies, not a **Compound**; the **nonconvergent** payload carries a chain and an alignment, not a **Path**. $\square$

6.2 The alternative interface cannot express this

Theorem 6.4 (Four-way conflation). *Let I be an interface returning a honjo value or nothing, with failure signalled by the absence of a value and no other channel. Then there exist four inputs, one each for *unclosed*, *inert*, *nonconvergent* and *underprovenanced*, on which I returns the same value. No property of a caller of I distinguishes them.*

Proof. Construct the four. *Unclosed*: `close N(H,H)`—nitrogen has $\nu = 3$ and two hydrogens close only two of them, so closure fails and no `Compound` exists. *Inert*: `close Ne(H)`—neon has $\nu = 0$, offers nothing to an interface, and again no `Compound` exists. *Nonconvergent*: a `track` whose chain does not admit, which by Theorem 7.4 yields no `Path`. *Underprovenanced*: an import under `require stated` of a graph containing supplied elements, which yields no `Graph`.

In all four cases I returns the absence of a value, and these are equal as results. A caller’s behaviour is a function of what it receives; equal results induce equal behaviour. Hence no caller distinguishes the four, and no discipline imposed on callers changes what is returned. The conflation is a property of I . $\square$

Corollary 6.5 (The repair is a wider return type). *No wrapper or convention applied to a value-or-nothing interface recovers the distinction, since by Theorem 6.4 the information is absent from the result. Recovering it requires the interface to return a verdict.*

Remark 6.6 (Inert is a positive claim). A verdict $(\text{inert}, \cdot)$ asserts that every participant is closed-shell, certified $\nu = 0$. It is therefore not interchangeable with $(\text{unclosed}, \cdot)$, which asserts the opposite about at least one participant. That the two are distinguishable is the chemical content of the discipline: “neon forms no compound” and “this attempt at ammonia was short two hydrogens” are different facts, and a language returning nothing for both has said neither.

7 Operational semantics

We give a small-step semantics over configurations $\langle \Gamma_G, M, \pi, e \rangle$, where Γ_G is the current contact graph (Theorem 2.1), $M \in \mathbb{N}$ is the committed-cut count (the program clock), π is the provenance map, and e is the expression under evaluation. Evaluation is measurement: steps mutate Γ_G and increment M .

7.1 Cut reduction

Rule 7.1 (Cut commits and counts).

$$\frac{S = \text{INDIVIDUATE}(\Gamma_G, Z) \quad \rho = w(\partial(S)) \geq \beta}{\langle \Gamma_G, M, \pi, \text{cut } Z \rangle \rightarrow \langle \Gamma_G \oplus \partial(S), M + 1, \pi, v_{\text{Atom}}(S, \rho, \text{stated}) \rangle}$$

where `INDIVIDUATE` is the framework’s atom-individuation operation (Section 8.1), $\Gamma_G \oplus \partial(S)$ is Γ_G with the cut’s boundary committed, and $v_{\text{Atom}}(S, \rho, p)$ is the resulting accountable atom value at provenance p .

The bond, deloc, close, and track rules are analogous: each commits boundary, increments M by the number of cut events it performs, and computes its value’s provenance as the maximum over its inputs (Theorem 2.7).

Theorem 7.2 (Cut monotonicity / intrinsic clock). *For any program, the committed-cut count M is non-decreasing along evaluation and strictly increases at every cut event. No reduction decreases M . Consequently re-evaluating the same surface expression is a new cut at a strictly higher count, never a cached recomputation, and two configurations with identical graphs but different M are distinct states.*

Proof. Every value-producing rule (Theorem 7.1 and its analogues) increments M ; no rule decrements it (there is no reduction that un-commits boundary—“undo” is itself a further cut). Hence M is monotone and strictly increases per cut event. Distinctness of equal-graph states at different M is immediate since M is part of the configuration. $\square$

Corollary 7.3 (Measurement, not simulation). *Because evaluation mutates Γ_G and advances M , a honjo cut cannot be a pure function memoised across calls: each occurrence performs the individuation afresh and is recorded in the clock.*

7.2 Track reduction and convergence

Rule 7.4 (Track is a residue chain).

$$\frac{(S_1, \dots, S_k) = \text{PROPAGATE}(\Gamma_G, x, P) \quad \rho_i = w(\partial(S_i)) \quad \text{ADMIT}(\rho_1, \dots, \rho_k) = \text{admit}}{\langle \Gamma_G, M, \pi, \text{track } x \text{ in } P \text{ until admit yield } r \rangle \rightarrow \langle \Gamma'_G, M + k, \pi, v_{\text{Path}}(S_{1:k}, \sum_i \rho_i, \max(p_x, p_P)) \rangle}$$

PROPAGATE is the framework’s accountable-propagation operation; each ρ_i is the residue of cut i and the boundary condition of cut $i+1$. ADMIT tests the convergence criterion. If the chain does not admit, the reduction instead yields the verdict (**nonconvergent**, $(S_{1:k}, \text{terminal alignment})$) by Theorem 6.2(V5), and the cut count still advances by k : the cuts were performed.

Remark 7.5 (A failed track still costs its cuts). The clause that M advances by k even on a **nonconvergent** verdict is not bookkeeping pedantry. The cuts were committed; by Theorem 7.2 the count cannot decrease; and a semantics that rolled the count back on failure would make a failed track free, which contradicts Theorem 2.11. A program that tries and fails has measured, and the clock records it.

Remark 7.6 (Local steps are free; only convergence is judged). In keeping with the underlying propagation theory, the intermediate cuts of a track are unconstrained: a step may be locally extreme provided the chain composes to a value meeting the admissibility criterion. ADMIT therefore inspects only the terminal convergence of the chain, not the plausibility of intermediate ρ_i . This is what makes **track ...until converge** a search over admissible propagations rather than a single forced path.

8 The standard library

Each honjo verb binds to a named operation of the underlying framework. We state the contract (inputs, output, the operation it invokes); the operation’s construction is external to this document.

8.1 cut — atom individuation

Definition 8.1 (INDIVIDUATE). **cut** Z invokes INDIVIDUATE(Z): from atomic number $Z \in \mathbb{N}^+$, fill the partition coordinates (n, ℓ, m, s) under the shell capacity $C(n) = 2n^2$, exclusion, and energy ordering, returning an **Atom** value carrying the configuration, the valence-shell occupancy q_v and capacity C_v , the vacancy $\nu = C_v - q_v$, the ground term symbol, and the residue $\rho \geq \beta$. E.g. **cut** 6 yields carbon $[\text{He}]2s^22p^2$, $\nu = 4$.

8.2 $\sim$ — bonding with committed cells

Definition 8.2 (BOND). **a $\sim$ b** : **k** invokes BOND(a, b, k): form the shared-boundary cut between items a, b committing k valence cells, and return a **Bond** value iff the shared content $\Delta\text{thick}(a, b) > 0$ and $k \leq \min(\nu_a, \nu_b)$. A closed-shell participant ($\nu = 0$) offers no cell, so no bond exists and the verdict is (**inert**, $\cdot$) (Theorem 6.2(V3)). The value carries k as its committed cell count.

Remark 8.3 (What the cell count is and is not). k records how many valence cells the interface commits, which is the quantity a separation must sever, and it coincides with the conventional bond order for ordinary multiple bonds (Lewis, 1916). It is not a bond energy and the framework does not supply one: the operation decides *whether* an interface lowers thickness and *how many* cells it commits, not by how much. Section 13 records this.

8.3 deloc — delocalised systems

Definition 8.4 (DELOC). `deloc ring( $a_1, \dots, a_n$ ) cells:  $c$` invokes DELOC($a_1, \dots, a_n; c$): form a single delocalised system over the n participants committing c cells in total, returning a Deloc value carrying the participant set, the total c , and the residue of the system’s cut against the medium. The value does *not* carry a per-pair cell count.

Proposition 8.5 (Delocalisation is not a per-bond attribute). *Let R be a delocalised system over n participants committing c cells in total. There is in general no assignment of integer cell counts to the individual pairs of R that both sums to c and is determined by the system. Consequently a representation assigning each pair a count states something the system does not.*

Proof. Take $n = 6$ and $c = 9$ (six σ interfaces and three delocalised cells). Any assignment $(c_1, \dots, c_6)$ with $c_i \in \{1, 2\}$ and $\sum_i c_i = 9$ requires exactly three $c_i = 2$, and there are two such assignments related by rotation. Both satisfy the total; the system determines neither, so selecting one asserts a fact about which pairs carry the extra cells that the system does not contain. Assigning fractional counts $c_i = 1.5$ instead states that each *pair* commits one and a half cells, whereas the system’s claim is about the ring. $\square$

Theorem 8.5 is why Theorem 8.4 carries a total and a participant set rather than a per-pair count, and why the grammar (Theorem 4.1) gives `deloc` its own form rather than treating it as sugar for a bond with a fractional annotation.

8.4 close — compound by closure

Definition 8.6 (CLOSE). `close X( $a_1, \dots, a_k$ )` invokes CLOSE: form the set of interfaces that drives every constituent’s vacancy to zero, returning a Compound value carrying the stoichiometry (fixed by vacancy matching), the per-centre geometry (maximal angular separation of the committed interfaces, compressed by unshared regions (Gillespie and Nyholm, 1957)), and total residue. If closure cannot be achieved the verdict is `(unclosed,  $\cdot$ )` naming the constituents with $\nu > 0$; if every participant is already closed the verdict is `(inert,  $\cdot$ )`.

8.5 track — accountable propagation

Definition 8.7 (PROPAGATE, ADMIT). `track x in P until converge yield r` invokes PROPAGATE(x, P): track item x ’s individuation through process P as a residue-chained sequence of cuts, and ADMIT the chain iff it converges to the output (the only admissibility test; local steps free, Theorem 7.6). The yielded Path is the accountable set of cuts by which x reached the output. Optional `with reps  $R$` permits representation switching between steps at no additional cut cost. A non-admitting chain yields `(nonconvergent,  $\cdot$ )`.

Remark 8.8 (The verbs are one verb). INDIVIDUATE, BOND, DELOC, CLOSE, and PROPAGATE are the cut at arities one, two, many-at-once, closure, and chain. The standard library is therefore not five independent operations but one operation exposed at five arities.

9 Importing structures

honjo generates atoms from atomic number. Real structures arrive from a converter as contact graphs with provenance. This section specifies the boundary.

Definition 9.1 (Graph value). A Graph value carries a contact graph Γ (Theorem 2.1), a provenance map π (Theorem 2.6), and the derived attribute

$$\text{supplied} = \frac{|\{x : \pi(x) = \text{supplied}\}|}{|\{\text{item vertices and contacts of } \Gamma\}|} \in [0, 1],$$

the fraction of the structure's own elements that were supplied by convention. Medium edges are excluded from the denominator: no record states them, so including them would make **supplied** a property of the representation rather than of the source.

Rule 9.2 (Import).

$$\frac{\Gamma \text{ satisfies Theorem 2.1 and } w(e) \geq \beta_\Gamma \ \forall e \quad \text{MEETS}(\pi, R)}{\Gamma \vdash \text{import graph } s \text{ require } R : \text{Graph} @ \beta_\Gamma / \max_x \pi(x)}$$

where $\text{MEETS}(\pi, R)$ holds iff: $R = \text{stated}$ and every element is **stated**; or $R = \text{supplied} < \varepsilon$ and the graph's **supplied** attribute is below ε . If the graph violates the floor condition the verdict is (**subfloor**, $\cdot$); if it fails MEETS the verdict is (**underprovenanced**, $\cdot$) carrying R and the measured fraction.

Proposition 9.3 (Import cannot upgrade provenance). *An imported Graph value has provenance stated only if every element of the imported graph is stated. No honjo operation applied to a supplied graph yields a stated value.*

Proof. The conclusion of Theorem 9.2 takes the maximum over all elements' tags, which is **stated** only if all are. The second claim is Theorem 5.8. $\square$

Remark 9.4 (Why the requirement is written in the program). The **require** clause is part of the import expression rather than a runtime check because a program's tolerance for supplied data is a property of the analysis, not of the data: an analysis of gross connectivity may accept a heavily supplied graph, while one turning on committed cell counts may not. Writing it in the source makes the tolerance reviewable and makes a violation a named verdict rather than a silent proceed (Fourches et al., 2010).

Remark 9.5 (What honjo does not do at the boundary). honjo does not read files, parse notations, perceive rings, or infer bond orders. It consumes a graph already carrying those decisions with their provenance. If a converter supplied a bond order, honjo will compute with it and will report the result as **supplied**; it will not second-guess the convention, and it has no basis on which to do so.

10 Core IR and the two-target model

10.1 The core IR

Definition 10.1 (Cut-IR). *Cut-IR* is a typed sequence of cut instructions over a contact-graph state. Each instruction is one of: $\text{IND}(Z)$ (individuate), $\text{BND}(a, b, k, g)$ (guarded bond committing k cells), $\text{DLC}(\bar{a}, c)$ (delocalised system), $\text{CLS}(X, \bar{a})$ (close), $\text{PRP}(x, P, \text{adm})$ (propagate), $\text{IMP}(s, R)$ (import under requirement), $\text{OBS}(v)$ (force measurement). Each instruction carries its result type, floor and provenance and, when executed, returns a verdict and increments the cut count. Cut-IR is back-end-agnostic: it names cuts, not how they are realised.

The front end (lexer, parser, type checker) is shared by both targets and lowers a program to Cut-IR. Only the realisation of a cut instruction differs between targets.

10.2 Target R (Rust): cuts as minimum cuts

Definition 10.2 (Rust realisation). Each Cut-IR instruction is realised by an exact computation on the contact graph: IND, BND, DLC, CLS compute the relevant minimum cut by maximum flow (Cormen et al., 2009; Ford and Fulkerson, 1956; Stoer and Wagner, 1997); PRP evaluates admissible propagations; residues are edge-weight sums in host floating-point (Goldberg, 1991). This is the reference semantics.

10.3 Target T (TypeScript/WebGL): cuts as render passes

Definition 10.3 (TypeScript realisation). Each Cut-IR instruction is realised as a GPU render pass over the contact-graph state encoded in textures: a fragment shader evaluates the cut at each texel and writes the framebuffer, whose contents are the result. Residues are read back from the framebuffer.

Definition 10.4 (Numeric resolution of a target). The *numeric resolution* ε_T of a target is the largest absolute error the target can introduce in computing a residue: for a floating-point back end, the accumulated rounding error over the cut’s edge set; for a render-pass back end, the sum of that and the framebuffer’s quantisation step.

10.4 Target equivalence, conditionally

Theorem 10.5 (Two-target equivalence up to the floor). *Let p be a well-typed honjo program with ambient floor β , and let v_R, v_T be the values it yields under the two back ends. Suppose $\varepsilon_R + \varepsilon_T \leq \beta$ (Theorem 10.4). Then v_R and v_T have the same type, provenance and cut structure, and their residues agree up to the floor:*

$$|\rho(v_R) - \rho(v_T)| \leq \beta.$$

Under that hypothesis the two back ends do not disagree about any distinction honjo can express.

Proof. Both back ends realise the same Cut-IR (Theorem 10.1), so they perform the same instruction sequence with the same types, provenances and cut count: the front end is shared, and type and provenance are computed there. They differ only in how each residue is computed. Writing ρ^* for the exact value, $|\rho(v_R) - \rho^*| \leq \varepsilon_R$ and $|\rho(v_T) - \rho^*| \leq \varepsilon_T$ by Theorem 10.4, so by the triangle inequality $|\rho(v_R) - \rho(v_T)| \leq \varepsilon_R + \varepsilon_T \leq \beta$ by hypothesis. Since honjo admits no sub-floor distinction (Theorem 5.4), the two results are observationally identical at the language level. $\square$

Remark 10.6 (The hypothesis is a real condition, not a formality). An earlier statement of this theorem asserted the conclusion unconditionally, on the reasoning that a target’s readback error is “bounded by the floor because the floor is the coarsest resolvable boundary”. That reasoning is wrong. β is a property of the contact graph’s weights, chosen by the program; ε_T is a property of the target’s arithmetic and framebuffer format, fixed by the hardware. They are independent quantities, and a half-float render target evaluating a program with a small β can have $\varepsilon_T > \beta$, in which case the two back ends can disagree about a distinction the language does express.

The hypothesis $\varepsilon_R + \varepsilon_T \leq \beta$ is therefore load-bearing, and the specification requires it to be *checked* rather than assumed: an implementation must report ε_T for its configuration, and a program whose ambient floor is below $\varepsilon_R + \varepsilon_T$ must be refused with the verdict (`subfloor`, $(\beta, \varepsilon_R + \varepsilon_T)$) rather than run. Section 12 makes this a conformance requirement.

Corollary 10.7 (Refusal rather than silent disagreement). *A program whose ambient floor is finer than the target’s numeric resolution is refused before execution, with both quantities named. No program runs on a target that cannot resolve its floor.*

Proof. By Theorem 10.6 the check is a comparison of two reported numbers, performed at program start. Failing it yields (`subfloor`, $\cdot$) by Theorem 6.2(V1), which by Theorem 6.3 carries no value. $\square$

11 Worked programs

11.1 Generate an atom

Listing 1: Individuating carbon (a length-one cut).

```
-- carbon.hj
floor 1.0 -- set the ambient floor; nothing is free

C := cut 6 -- individuate Z=6 against the medium
observe C -- force the measurement now
-- C : Atom @ 1.0 / stated
-- config [He] 2s2 2p2
-- vacancy 4
```

11.2 Build a compound, with committed cells

Listing 2: Water and carbon dioxide. The cell count is what distinguishes a double interface from two singles.

```
-- oxides.hj
floor 1.0

O := cut 8
H := cut 1
C := cut 6

-- single interfaces: one committed cell each
W := close O(H, H) -- 2:1, bent
observe W

-- carbon dioxide: each C=O commits two cells
CO2 := close C(O : 2, O : 2)
observe CO2
assert CO2.vacancy == 0 emit "CO2 did not close"

-- neon offers no cell: the verdict is inert, not an empty compound
Ne := cut 10
X := close Ne(H) -- verdict: inert, certifying nu = 0
```

11.3 A delocalised system

Listing 3: Benzene: a ring, not six pairwise bonds.

```
-- benzene.hj
floor 1.0

c1 := cut 6
c2 := cut 6
c3 := cut 6
c4 := cut 6
c5 := cut 6
c6 := cut 6

-- one delocalised system over six centres, nine cells in total.
-- deliberately NOT six bonds at 1.5 cells each (Prop. deloc).
ring := deloc ring(c1, c2, c3, c4, c5, c6) cells: 9
```

```
observe ring
```

11.4 Import a structure and require its provenance

Listing 4: A program that will not compute over convention-heavy input.

```
-- imported.hj
floor 1.0

-- the analysis turns on committed cell counts, so it will not accept a
-- structure whose cell counts were largely supplied
g := import graph "target.graph"
    require supplied < 0.10
    unless refuse

path := track g in g
    until converge
    yield amalgamation

observe path
-- path : Path @ 1.0 / supplied <- provenance survives the track
```

Remark 11.1 (Reading the last line). Even though the import met its requirement, the imported graph contained *some* supplied elements, so by Theorem 5.7 the tracked path is supplied. The `supplied < 0.10` attribute and the `supplied` tag answer different questions: the attribute says how much of the input was convention, the tag says that the result depends on convention at all. Theorem 5.9 explains why the language carries both.

12 Implementation requirements

Both targets must implement, from this specification:

1. The shared front end: lexer (Section 3), parser to the grammar (Theorem 4.1), and accountability type checker (Section 5) enforcing Theorem 5.4 and Theorem 5.7.
2. Lowering to Cut-IR (Theorem 10.1).
3. The verdict discipline of Section 6: every operation that can fail returns a labelled verdict, and no failure verdict carries a value (Theorem 6.3).
4. The operational semantics of Section 7: contact-graph state, monotone cut count, provenance propagation, and the cut/bond/deloc/close/track reductions.
5. The standard-library bindings of Section 8 and the import rule of Section 9.
6. A back end (Theorem 10.2 or Theorem 10.3), which must **report its numeric resolution** ε_T (Theorem 10.4) and refuse any program whose ambient floor is below $\varepsilon_R + \varepsilon_T$ (Theorem 10.7).

12.1 Conformance suite

A conforming implementation must pass:

- (C1) **No sharp cut.** No program produces a value with $\rho < \beta$; attempts are rejected at type-check.
- (C2) **Provenance monotonicity.** For a corpus of imported graphs with mixed tags, every derived value's tag equals the maximum over its inputs, and no `supplied` input yields a stated output.

- (C3) **Six verdicts realised.** Each label of Theorem 6.1 is produced by some program in the suite, and no failure verdict carries a value.
- (C4) **Conflation control.** The four programs of Theorem 6.4 produce four distinct labels; run through a deliberately impoverished value-or-nothing wrapper, they produce one result, confirming the theorem is not vacuous.
- (C5) **Cell counts distinguish.** $C \sim 0 : 2$ and two $C \sim 0 : 1$ interfaces produce different values.
- (C6) **Delocalisation is not a count.** A `deloc ring` value carries no per-pair count, and no program extracts one.
- (C7) **Resolution gate.** A program whose floor is below the target’s reported ε_T is refused, with both numbers in the verdict.
- (C8) **Target agreement, conditionally.** For every program passing (C7), the two back ends agree in type, provenance and structure, with residues within β . Programs failing (C7) are *not* run on both targets and are recorded as out of scope, since Theorem 10.5 does not apply to them.

Remark 12.1 (Why (C8) excludes rather than tolerates). A conformance suite that ran sub-resolution programs on both targets and reported the disagreement as tolerable would be asserting the unconditional theorem that Theorem 10.6 retracts. Excluding them is the honest form: the equivalence holds where its hypothesis holds, and the suite records which programs those are.

13 What honjo does not do

- L1. No bond energetics.** BOND decides whether an interface lowers boundary thickness and how many cells it commits, not by how much. honjo therefore cannot rank two candidate structures by stability, and a program that appears to do so is reading a residue, which is a separation cost, not an energy (Theorem 8.3).
- L2. No covalent/ionic distinction.** The underlying operation treats an interface through vacancy sharing alone, so a bond in which the shared cells are held very unequally and one in which they are held evenly are the same object to honjo. This is a limitation of the chemistry the language binds to, not of the language.
- L3. Multiple bonding is counted, not adjudicated.** The cell count records how many cells an interface commits. Nothing in honjo decides *whether* a double interface or two singles is the realised structure; a program must state which it means. `deloc` is likewise a declaration, not a perception.
- L4. No file input, no perception.** honjo does not read records, parse notations, or perceive rings, aromaticity or stereochemistry. Structures arrive already translated (Theorem 9.5).
- L5. Provenance is two-valued and coarse.** The tag distinguishes stated from supplied and nothing finer: it does not record *which* convention supplied an element, nor weight elements by importance (Theorem 5.9). A supplied bond order at a reactive centre and a supplied aliphatic hydrogen both make a result supplied.
- L6. Soundness is proved as a sketch.** Theorem 5.10 is argued, not formalised, and the mutation clause is the non-routine step (Theorem 5.11). A mechanised proof would be worth having and we have not carried one out.
- L7. Target equivalence is conditional and the condition is measured, not derived.** Theorem 10.5 holds when $\varepsilon_R + \varepsilon_T \leq \beta$. Whether a given GPU configuration satisfies

this is an empirical property of that hardware which the implementation must report; the specification requires the check but cannot discharge it (Theorem 10.6).

- L8. No performance claim.** Nothing here asserts that the render-pass realisation is faster than the exact one for any particular program, only that it is a distinct realisation of the same IR.

14 Conclusion

We have specified Honjo Masamune as a language whose single primitive is the cut—the act of individuating a part of a bounded whole at the irreducible cost $\beta > 0$ —and whose every operation, from generating an atom to tracking an item through a process, is that primitive at a different arity.

Three properties are enforced statically. No value may claim zero residue (Theorem 5.4), so the sharp cut is untypeable. No operation may launder supplied data into stated data (Theorems 5.7 and 5.8), so an audit performed before honjo survives every computation after it. And no operation that can fail returns an empty value: it returns a labelled verdict, of which only one label carries a value (Theorem 6.3), because a value-or-nothing interface provably conflates four distinguishable failures that no caller discipline recovers (Theorem 6.4).

The surface language gained two forms it previously lacked and needed: committed-cell counts on bonds, without which multiply-bonded structures are inexpressible; and a delocalised-system form which is not sugar for fractional per-bond counts, because Theorem 8.5 shows such counts state what the system does not. Structures enter through a typed import that refuses rather than silently accepting a graph whose provenance falls short of what the program declared it needs.

One earlier claim has been retracted and replaced. Two-target equivalence was previously asserted unconditionally on the reasoning that a target’s readback error is bounded by the floor; that reasoning conflates a property of the program’s weights with a property of the hardware’s arithmetic. Theorem 10.5 now carries the hypothesis $\varepsilon_R + \varepsilon_T \leq \beta$, the implementation must report ε_T , and a program whose floor is finer than its target can resolve is refused rather than run (Theorem 10.7). The conformance suite records which programs the equivalence covers instead of tolerating disagreement on the ones it does not.

Section 13 states eight things the language does not do, three of which are limitations of the chemistry it binds to and cannot repair from inside. A specification whose worked examples all succeed has not been shown capable of refusing anything, and refusal is a property this one claims: Section 11 includes a program that declines to compute, and Section 12 requires a conformance check that it does.

References

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956. doi: 10.4153/CJM-1956-045-5.
- Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify: On the importance of chemical structure curation in cheminformatics and QSAR modeling research. *Journal of Chemical Information and Modeling*, 50(7):1189–1204, 2010. doi: 10.1021/ci100176x.
- Ronald J. Gillespie and Ronald S. Nyholm. Inorganic stereochemistry. *Quarterly Reviews, Chemical Society*, 11(4):339–380, 1957. doi: 10.1039/QR9571100339.
- David Goldberg. *What Every Computer Scientist Should Know About Floating-Point Arithmetic*. ACM Computing Surveys 23(1), 5–48, 1991.

- International Organization for Standardization. *ISO/IEC 14977: Information technology — Syntactic metalanguage — Extended BNF*. ISO, 1996.
- Gilbert N. Lewis. The atom and the molecule. *Journal of the American Chemical Society*, 38(4): 762–785, 1916. doi: 10.1021/ja02261a002.
- Robin Milner. A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3):348–375, 1978. doi: 10.1016/0022-0000(78)90014-4.
- B. C. Pierce. *Types and Programming Languages*. MIT Press, Cambridge, MA, 2002.
- Mechthild Stoer and Frank Wagner. A simple min-cut algorithm. *Journal of the ACM*, 44(4): 585–591, 1997. doi: 10.1145/263867.263872.